Research Square

Preprints are preliminary reports that have not undergone peer review. They should not be considered conclusive, used to inform clinical practice, or referenced by the media as validated information.

Integrated Technique of Natural and Artificial Texts Authorship Verification in the Academic Environment

Aleksandr Romanov

alexx.romanov@gmail.com

Tomsk State University of Control Systems and Radioelectronics

Alexander Shelupanov

Tomsk State University of Control Systems and Radioelectronics

Anna Kurtukova

Tomsk State University of Control Systems and Radioelectronics

Anastasiia Fedotova

Tomsk State University of Control Systems and Radioelectronics

Research Article

Keywords: plagiarism detection, educational process estimation, artificial generation, source code, authorship attribution, authorship verification

Posted Date: September 16th, 2024

DOI: https://doi.org/10.21203/rs.3.rs-4914352/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License. Read Full License

Additional Declarations: No competing interests reported.

Integrated Technique of Natural and Artificial Texts Authorship Verification in the Academic Environment

Aleksandr Romanov[0000-0002-2587-2222], Anna Kurtukova [0000-0001-5619-1836] , Anastasiia Fedotova [0000-0001-7844-4363] and Alexander Shelupanov [0000-0003-2393-6701]

[1] Department of Security, Tomsk State University of Control Systems and Radioelectronics, 634050 Tomsk, Russia
alexx.romanov@gmail.com

Abstract. The issue of text plagiarism in academic and educational environments is becoming increasingly relevant every year. The quality of research articles and works is declining due to students copying fragments of others' works and using modern generative models for text and source code creation. The article proposes an integrated technique for authorship verification of both natural and artificial language texts, based on a combination of statistical methods, machine learning, and deep neural networks. The presented technique addresses several related tasks: assessing text homogeneity, detecting plagiarism when solving closed-set authorship attribution problems, and identifying texts and fragments created by generative models. Experimental data include a multi-domain dataset of natural language texts consisting of research articles on natural and technical sciences, PhD dissertations, and artificially generated texts on related topics. To evaluate the effectiveness of the technique in relation to artificial language texts, a multilingual program dataset was used, consisting of source codes for programs of technical students as well as artificially generated program codes. The experimental results demonstrate the effectiveness of the proposed technique for plagiarism detection and copyright protection in the educational process. The accuracy of identifying heterogeneous fragments in text or code is 93-94%, authorship attribution ac-curacy is 89-99% depending on the number of co-authors, and verification accuracy is 97.5-99.4%.

Keywords: plagiarism detection, educational process estimation, artificial generation, source code, authorship attribution, authorship verification.

1 Introduction

Plagiarism and artificial generation in the educational process are crucial problems that need attention. Academic borrowings in the educational process harm both personal and professional development, as well as the scientific community as a whole. Plagiarism violates academic ethics, decreases the benefits and value of education, has negative impacts on careers, and creates undue risks for future scientific research [1].

It is impossible to imagine the modern educational process and academic environment without technologies designed to increase the objectivity of assessment and detect plagiarism. Among these technologies, the most known are:

Plagiarism checking systems [2-4] are specialized programs and services designed to check texts for plagiarism. The principal focus of their work is text analysis and comparison with databases, internet sources, and others to detect non-original content.

Copyright systems [5, 6] are systems designed to manage copyrights to protect authors' intellectual property and prevent their theft.

Technologies for recognizing artificially generated content [7, 8] are methods and models designed to detect text or code created by a generative neural network capable of adapting to the style of a specific author.

Most of these tools are based on algorithms for detecting fuzzy text duplicates, as well as machine learning and deep learning.

The study [9] presents an approach to detecting plagiarism in texts based on optimized differential evolution embeddings, long short-term memory (LSTM), and Bidirectional Encoder Representations from Transformers (BERT) using focal loss. The datasets include the Microsoft Research Paraphrase Corpus (MSRP), the Semantic Evaluation Database (SemEval), and the Stanford Natural Language Inference (SNLI). The accuracy of the approach for the MSRP dataset is 0.93, with an F-measure of 0.95; for the SemEval dataset, the accuracy is 0.87, with an F-measure of 0.92; and for the SNLI dataset, the accuracy is 0.93, with an F-measure of 0.9. According to the authors' conclusions, this approach is more effective than RNN-based approaches [10], Siamese networks and LSTM [11], and the method based on Siamese networks, bidirectional LSTM, and attention mechanisms [12].

The authors [13] propose a plagiarism detection system based on the chi-square technique and support vector machine (SVM) method, trained on a set of informative features. The approach is evaluated using the PAN-2013 and PAN-2014 competition datasets. In the experiment, the authors test various sets of features. The best result is demonstrated by SVM trained on 32 informative features obtained through lexical, syntactic, and semantic analysis. The F-measure for PAN-2013 is 0.89, and for PAN-2014 it is 0.93.

The approach presented in [14] is based on three deep models: doc2vec, Siamese LSTM (SLSTM), and Convolutional Neural Network (CNN). The first step involves transforming the documents into vector form using doc2vec. Then, SLSTM is trained to create representations adapted to different types of plagiarism. CNN is employed to provide a binary answer indicating whether the document is plagiarized or not for the actual plagiarism detection. The approach was evaluated on the PAN dataset, which is specifically annotated for plagiarism detection. The achieved accuracy is 0.98.

An approach to plagiarism detection based on semantic knowledge is proposed in the article [15]. The approach uses the lexical database WordNet and the Dice measure as a similarity metric for pairs of sentences to understand the context. Additionally, path similarity measures and depth evaluations with different weights are used as sources of information for plagiarism detection. The approach was evaluated using the PAN-11 dataset, achieving an F-measure of 0.875. The authors claim that this approach is robust against paraphrasing, synonymization, and restructuring.

The authors of the article [16] present an approach to detecting plagiarism in source code. The approach involves several stages. First, the source code is transformed into contextual embeddings that reflect the syntax and semantics of the program text. These representations are generated using the pre-trained model CodePTMs. Next, cosine similarity scores are calculated between pairs of code snippets. The classifier is selected automatically using the AutoML tool. The chosen classifier predicts probabilities, which are then compared to a dynamic threshold to make a final decision about the presence or absence of plagiarism. The approach was evaluated using the SOCO competition dataset for C++ and Java languages. The accuracy of the presented approach reaches up to 0.97, depending on the usage scenario.

The advantages of modern approaches include high accuracy even in cases of deliberate paraphrasing, restructuring, and misleading, as well as high operation time and broad coverage of sources. The disadvantages of the approaches include the fact that their effectiveness on content containing different domains is unproven, as well as their inapplicability to distinguishing authorship between humans and machines. Moreover, these approaches do not address the entire spectrum of issues associated with text plagiarism. Instead, they focus on detecting matches of text or code fragments according to a knowledge base.

One of the methods to prevent plagiarism and academic borrowings, addressing the shortcomings and limitations described, could be authorship identification systems that consider an open set of candidates. This method enhances the capabilities of text analysis, plagiarism detection, and distinguishing authorship between humans and machines due to the following factors:

- Considering stylistic and linguistic features. Plagiarism detection systems can be ineffective when a text has been paraphrased or changed, or they may generate false positives. Analyzing the stylistic and linguistic features of a text is helpful to identify unique traits of the author's style and improve the efficiency of false-positive detection.
- Deep text analysis. Approaches based on machine learning and statistical methods enable a more in-depth analysis of text, taking into account various aspects such as word frequency, grammatical features, and other structural elements. This allows for the identification of more latent differences between texts.
- Adaptability to changes. Approaches based on machine learning can be more adaptive to changes in the text or the author's style.

Based on this, a pressing issue for the academic and educational environment is the development of a comprehensive technique for detecting plagiarism and artificially generated text through authorship attribution methods. The scientific novelty of the research lies in the development of a technique that distinguishes itself through the use of a Siamese neural network (SimNN) trained with contrastive and triplet loss functions, alongside an ensemble of distance measures. This technique addresses for the first time the problem of detecting plagiarism in cases involving an open set of candidate authors, taking into account artificial text generation.

The article is organized as follows: Section 2 is dedicated to our previous research on the authorship of natural and artificial language texts; Section 3 focuses on the

process of collecting, generating, and forming experimental data; Section 4 presents the technique for authorship attribution of natural and artificial language texts, aimed at detecting plagiarism and artificially generated text fragments; Section 5 outlines the setup and execution of experiments designed to confirm the effectiveness of the proposed technique; and Section 6 is the conclusion and provides a summary of the article.

2 Our Previous Research

In the course of previous research, effective and universal techniques for identifying the authorship of natural and artificial language texts were developed based on advanced statistical algorithms, machine learning, and deep learning techniques. The earlier studies addressed the following cases.

Authorship attribution of Russian-language text's author regardless of their domain: fanfiction, scientific, literary, destructive, or short messages on social networks [17, 18]. This task requires a comprehensive approach, which includes:

- careful preprocessing including data filtration;
- informative feature extraction;
- analysis of the text at syntactic, lexical, morphological, and especially semantic levels;
- selection of the most effective classifiers according to the specificity of the data in a given domain.

The classification accuracy of literary, fanfiction, and social media short texts using a combination of Gated Recurrent Units (GRU)+CNN reaches 98%. The highest classification accuracy for destructive texts is achieved through the application of semantic clustering and transfer learning, averaging 92%.

The identification of the source or disassembled code author [19, 20] relies on an ensemble of machine learning and deep learning algorithms: a proprietary hybrid neural network (HNN), SVM trained on expert-selected features, and fastText with experimentally optimized parameters. The weights of each classifier's decisions are adapted to the specific task. For source codes, the most effective classifier is HNN, achieving an average accuracy of 95% regardless of the programming language in simple cases and 85% in complex cases (e.g., obfuscation, coding standards, team development). Consequently, HNN's decisions are given the highest weight in authorship identification of source code. For disassembled code, the best results are achieved with fastText, which allows for an accuracy of 97%, thus giving its decision the highest priority in the voting process. This ensemble demonstrates high accuracy in solving real-world tasks and is resilient to various complicating factors.

3 Dataset Description

3.1 Natural language dataset

When creating the dataset, particular attention is paid to texts created in educational and academic environments. Such texts include research articles, technical reports, dissertations, PhD dissertations, essays, etc. As experimental data, it was decided to consider publicly available texts created by young scientists and degree candidates: PhD dissertations and research articles in Russian (see Table 1). The data were collected from open sources with high impact factors and citation indices. Articles were selected from technical, physical-mathematical, chemical, medical, and philological sciences (see Fig. 1). Most of the texts were obtained from publications such as «Annals of the Russian academy of medical sciences», «Vestnik NSU», «Tomsk State University Journals», «Bulletin of Moscow University», «Vestnik of Saint Petersburg University», «SPIIRAS Proceedings», and «Proceedings of TUSUR University». The set also included 40 research articles from our research group with pre-known fragments written by co-authors.

Based on the obtained dataset, additional texts for each topic were also generated using GPT-3.5 [21] and GPT-4 [22].

Table 1. Statistics of the natural language dataset

Characteristic	Research articles	PhD dissertations	GPT-3.5	GPT-4
Number of authors	3971	525	250	250
Number of texts	4500	525	1000	1000
Av. length of words per text	2713	31762	1521	1645
Av. length of symbols per text	21401	247743	12670	13688
Av. number of sentences per text	208	2443	163	180
Number of articles written by one author	281	–	–	–
Number of articles written by 2-5 co-authors	2317	–	–	–
Number of articles with >5 co-authors	1902	–	–	–

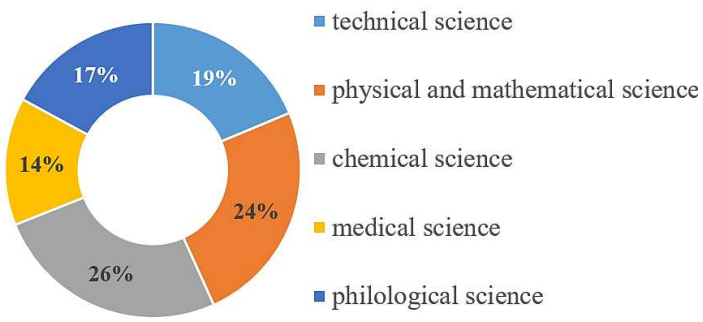

Fig. 1. Distribution of data by topic.

3.2 Artificial language dataset

The topic of academic borrowing and artificial generation in the creation of program code within the educational process is of particular interest, especially in the context of practical programming classes for students in technical specialties. Based on this, a dataset was formed as follows:

- source code written by students as part of practical programming classes;
- source code written by students as part of group and project-based development in programming and machine learning;
- source code written by our research team;
- source code generated by GPT-3 [21] and GPT-4 [22] models in accordance with the author's style.

The dataset includes source code written in C#, C++, Python, Java, Go, JavaScript, and Perl. The dataset statistics are presented in Table 2.

Table 2. Statistics of the artificial language dataset

Characteristic	Source codes	GPT-3.5	GPT-4
Number of authors	75	75	75
Number of files	600	600	600
Number of programming languages	7	7	7
Av. number of symbols per text	1714	1323	1452
Av. number of lines per text	129	109	118

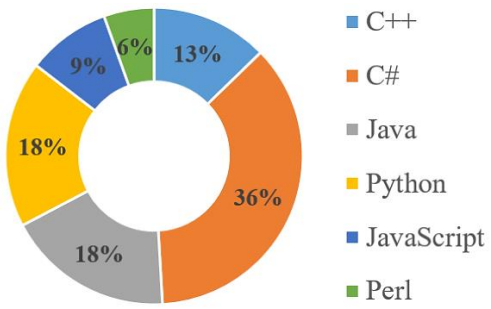

Fig. 2. Distribution of data by programming language.

4 Integrated technique for plagiarism detection in academic environments based on authorship attribution

Integrated technique of natural and artificial texts authorship verification in the academic environment is presented on Figure 3.

4.1 Data preprocessing

During the data preprocessing stage, domain identification, data cleansing, and segmentation are conducted to facilitate the evaluation of text homogeneity.

First and foremost, the classification of the text as either natural or artificial language is determined. The outcome of this assessment directly influences the selection of methods employed in subsequent stages.

For performing such binary classification, SVM trained on a diverse set of texts from various domains and a set of source codes written in different programming languages is utilized. The selection of SVM as the classifier is due to its high training and execution time, as well as its effectiveness in solving related tasks. The SVM was trained on a set of informative features selected by experts in our previous studies [17, 20]. The accuracy of this classifier reaches 99.2%.

After determining the text's domain, the cleaning process occurs. In the context of natural language texts, it involves:

- de-identification – removing any mentions of the articles/PhD dissertations author(s);
- removal of special characters, links, tags, etc.;
- removal of bibliographic information and acknowledgments.

For artificial language texts, the cleaning process consists of de-identification, which includes removing authorship comments and direct directory paths.

The final task of the preprocessing stage is the segmentation of the text into fragments. For natural language texts, segmentation is performed at punctuation marks indicating the end of a sentence. For artificial language texts, the segmentation is done by lines.

4.2 Assessment of homogeneity

In the homogeneity assessment stage, heterogeneous text fragments are identified. The proposed technique includes using QSUM combined with SVM as a time-oriented approach, and a method based on a Siamese neural network and ensemble measures as an accuracy-oriented.

For the described task, the QSUM method presented in [23] was modified and include the following steps:

1. Feature extraction for each sentence in the text. Keywords extracted by BERTopic were utilized as these features.
2. Calculation of the cumulative sum matrix $Q = (Q_{1,1},...,Q_{N,K})$:

$$Q_{i,j} = \sum_{i=1,j=1}^{N,K} (h_{i,j} - \overline{h}_j) \tag{1}$$

   where $Q_{i,j}$ is the cumulative sum value for the i-th fragment S_i for the j-th feature H_j, $h_{i,j}$ is the numerical value of the feature in the analyzed fragment S_i, $h_{i,j}$ is the numerical value of the feature in the analyzed fragment S_i, $\overline{h}_j$ is the average value of the j-th feature H_j across the entire text, and N is the number of fragments in the text.
3. Regression analysis. Then, the regression equation is calculated to determine the relationship between features for each pair of rows in the matrix **Q** and the standard error of the regression. The cumulative sum value is used as the independent variable.
4. Identification of potentially heterogeneous fragments. In this step, a matrix $G = (G_{1,1},...,G_{N,M})$, is formed, where Gi,j can take values of 0 and 1. A value of 0 indicates homogeneity, while a value of 1 indicates a probable heterogeneity of the text fragment Si, for which the residual error exceeds the standard error of the regression by a factor of $Q_{i,j} - Q'_{i,j}$ in T_1 times:

$$Se_j = \sqrt{\frac{\sum_{i=1}^{N} (Q_{i,j} - Q'_{i,j})^2}{N-2}} \tag{2}$$

where $Q_{i,j}$ is the actual cumulative sum value for the i-th fragment Si for the j-th feature Hj, and $Q'_{i,j}$ is the cumulative sum value calculated using the regression equation.

5. Voting. The previously obtained vectors are fed into a one-class SVM. In this case, the final number of positive classifications across all feature pairs is summed, forming a score vector. The decision regarding heterogeneity is made based on the majority vote.

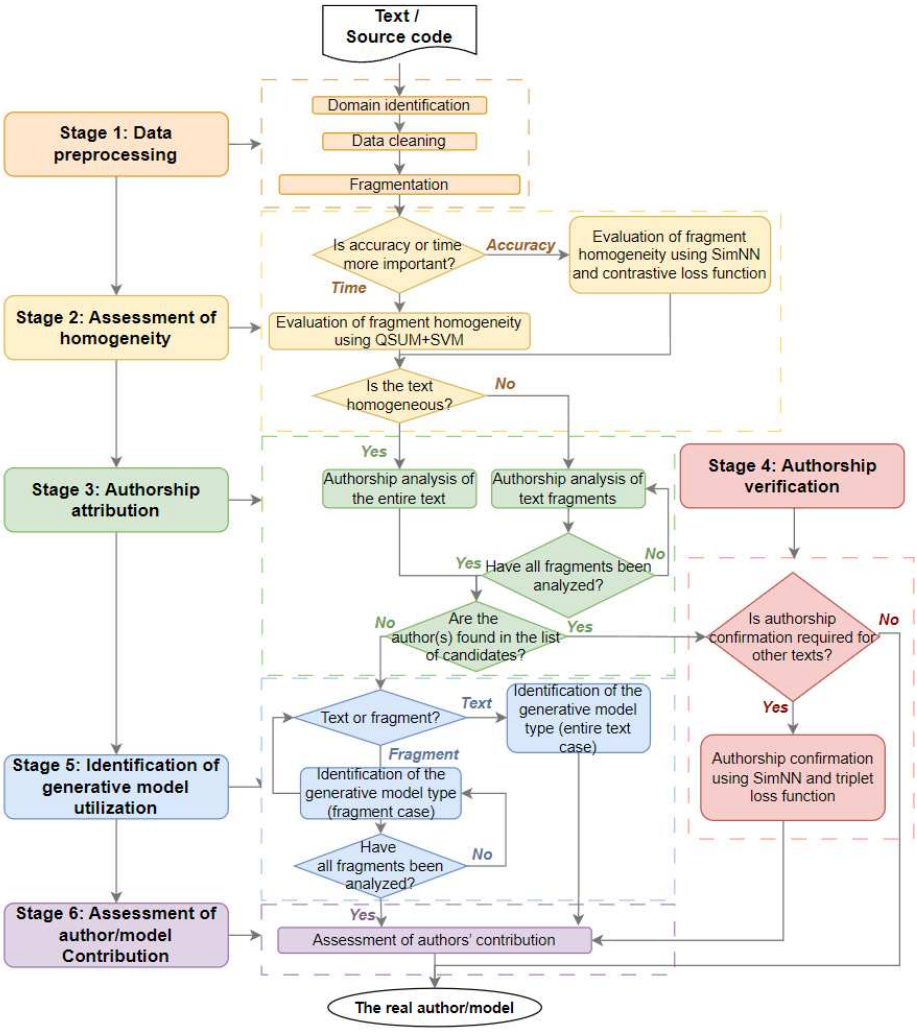

Fig. 3. Integrated technique of natural and artificial texts authorship verification in the academic environment

The accuracy-oriented approach is based on SimNN and an ensemble of distance measures, consisting of two main steps:

1. The foundation of effective author verification is the presence of a reference with reliable authorship and high-quality vector representations. The model used is based on contrastive learning, aiming to minimize differences between texts by the same author and maximize differences between texts by different authors. This model is SimNN [24, 25]. For effective training of SimNN, the use of specific loss functions – triplet (t_l) or contrastive (c_l) – is recommended. The unique feature of these loss

functions lies in training the model on a triplet consisting of an anchor (text), a positive example (text by the same author), and a negative example (text by a different author). The principle of the proposed approach for creating vector representations is illustrated in Figure 4.

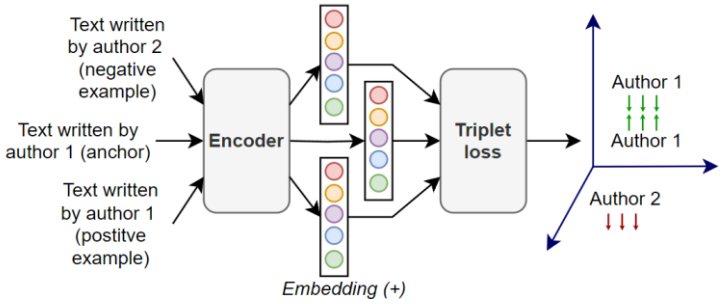

Fig. 4. An approach to creating embeddings.

2. Based on embeddings formed by the contrastive learning model, the final verification stage involves calculating the distance measure. To ensure the objectivity of the assessment and simplify its adaptation to the specific domains of textual data, an ensemble of cosine, Euclidean, Manhattan distances, and the Jaccard coefficient is used [26]. The averaging of the ensemble's assessments is performed using the median after Min-Max normalization, which helps to reduce the influence of outliers and anomalies. An author is confirmed when the averaged distance measure does not exceed the threshold distance. Conversely, if the threshold is exceeded, the analyzed texts/source codes are considered to belong to different authors and are deemed heterogeneous. In this method of assessing homogeneity, each fragment is compared with every other fragment.

4.3 Authorship attribution

Authorship attribution of natural language texts.
At the authorship attribution stage for heterogeneous fragments, the author is identified from an open set of candidates. This analysis allows for the possibility that the text or fragment was written by an author not included in the candidate set or generated by a generative neural network. The decision-making method for attributing a fragment to an author employs a combination of GRU and CNN (Fig. 5).

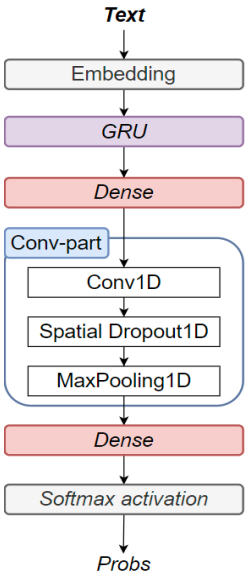

Fig. 5. Architecture of GRU+CNN.

The approach operates in several stages:

1. The input text is transformed into dense vectors of fixed dimensionality. The input dimensionality of the embedding is determined for each individual dataset and is set to the size of the vocabulary – the number of unique words. The output dimensionality, set to 200, has been selected based on experimental results.
2. The obtained representations are passed to fully connected (Dense) and recurrent GRU layers. These layers process sequential data to preserve contextual and sequential information, as well as to reduce their dimensionality.
3. The results obtained from the recurrent and Dense layers are passed to the convolutional part. The layers within this part are responsible for extracting high-level features from the sequential data, preventing overfitting, and reducing dimensionality. The operations performed at this stage make the model more invariant to small shifts in the input data.
4. The final layer is a Dense layer with a Softmax activation function, which enables the model to produce a probability distribution over all authors involved in the training process.

Authorship attribution of artificial texts.
The problem of identifying the author of a code/its fragment is addressed using an original technique proposed in the study [19]. Decision-making methods include an ensemble of HNN, fastText, and SVM with weighted voting (Fig. 6a), as well as CodeBERT with a Dense layer (Fig. 6b).

The first classifier in the ensemble is HNN, which includes convolutional, recurrent, and Dense layers. Experiments demonstrate that this architecture effectively captures

the unique features of source code as the object of study. The convolutional part, represented by Inception-v1, enables the model to focus on local patterns in the sequences of source code symbols. Its filters, with different sizes, extract patterns of lengths 1, 3, and 5. The recurrent part, represented by BiGRU, accounts for both preceding and succeeding contexts, allowing for an efficient analysis of the code structure. The Dense layer transforms the high-level features obtained from previous layers into a more compact representation. A distinctive feature of this model is its ability to autonomously extract both explicit and implicit authorial features from vector representations. According to the proposed technique, the training and predictions of the HNN are most effective when using one-hot encoding applied to raw, unprocessed source code. The HNN model is trained over 50 epochs.

The second classifier is SVM, trained on a set of features selected through expert analysis. This feature set includes structural, lexical, and «code smells» as proposed by the research group of Caliskan-Islam A. [27]. The number of epochs is determined by a stopping criterion.

The third classifier is fastText, with hyperparameters selected through a greedy search: a learning rate of 0.1, a rate of updates for the learning rate set at 100, a context window size of 5, 5 negative samples, a Softmax loss function, and 50 epochs.

Weights are applied to the results of each algorithm in the ensemble. These weights are determined based on the effectiveness of the models, as demonstrated in [19]. For source code, the optimal combination of weights is 0.4 for HNN, 0.3 for fastText and SVM, respectively. This weighting scheme prioritizes the decisions of HNN. In cases where the predictions of fastText and SVM are equal, their decision is given precedence.

In contrast to the ensemble, the CodeBERT model developed by Microsoft is chosen as a classifier. This model is a modification of the multilingual BERT, trained on source codes in various programming languages. Therefore, the model possesses specific knowledge about the features of source codes as the object of study and an efficient tokenization method that accounts for the syntax and structure features. Since CodeBERT in its original form functions as an encoder, its architecture was augmented with a linear layer, with the output dimension determined by the number of authors involved in the training. CodeBERT's hyperparameters were selected through a greedy search: a learning rate of 0.00003, a regularization coefficient of 0.01, a batch size of 16, and 10 training epochs.

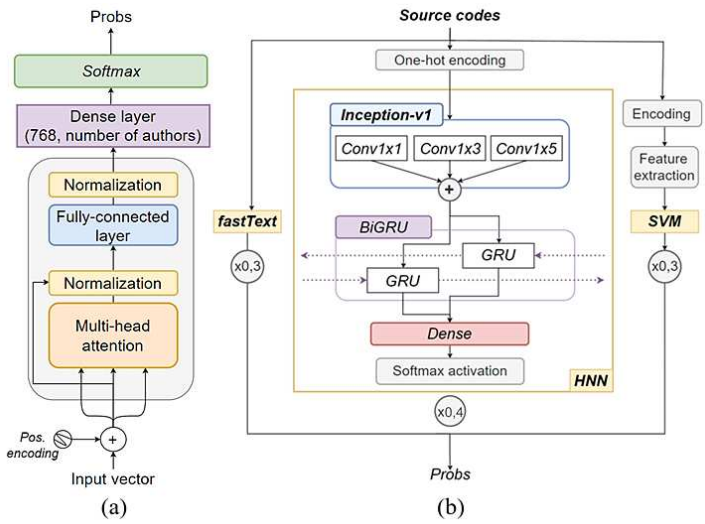

Fig. 6. Architecture of CodeBERT (a) and the ensemble (b).

4.4 Authorship verification

The author verification stage is conducted for texts that have undergone homogeneity checks. This stage aims to confirm that two or more texts belong to the same author. Author verification is carried out using SimNN and an ensemble of measures; however, complete texts with a reliably known author or model are used instead of sentences.

If the real author is not identified during the verification stage, an analysis is conducted to determine whether generative models were used in the creation of the text. This analysis employs a method based on GRU+CNN for natural language texts, using an ensemble of classifiers including HNN, SVM, and fastText. Conversely, for artificial languages, CodeBERT is utilized.

4.5 Evaluation of author contribution

The final stage of the technique involves evaluating the contribution of the author and/or the generative model in the creation of the analyzed text. This includes verifying the authorship of each fragment and summing the portions attributed to a specific author or model. The overall contribution is calculated as follows:

$$contribution = \frac{number_of_author's_fragments}{total_number_of_fragments} \times 100 \quad (3)$$

If the percentage of academic borrowings exceeds 30% of the total volume, the text is deemed non-original. This threshold is set in accordance with the requirements of Russian journals with high impact factors and citation indexes.

5 Experimental setup and results

To determine the quality of text homogeneity assessment, research articles published by our research group were used. The experiment compared the accuracy and time of homogeneity assessment of text fragments using QSUM+SVM and SimNN, trained with *t_l* and *c_l* losses, and applied an ensemble of measures. The experiments were conducted on fragments with varying numbers of sentences, ranging from 5 to 45, as the effectiveness of the approaches directly depends on the size of the analyzed fragments. The results are presented in Table 3.

Table 3. Accuracy assessment of detecting heterogeneity in natural language texts

Method	Number of sentences	Accuracy, %	Time, fragments/sec
	5	73.2	0.00007
	15	81	0.00032
QSUM + SVM	25	85.5	0.00068
	35	84.1	0.00099
	45	79.5	0.00105
	5	75,7	0.016
	15	86,5	0.045
SimNN+c_l	25	94	0.081
	35	92,7	0.098
	45	90	0.103
	5	75,2	0.012
	15	86,1	0.042
SimNN+t_l	25	93,3	0.08
	35	91	0.091
	45	89,4	0.099

To assess the quality of the homogeneity of source codes, Python, JS scripts, and C++ codes developed by our research group led by associate prof. A.S. Romanov were utilized. Given the infeasibility of adapting the SQUM approach to the source codes, it was decided to conduct experiments exclusively with SimNN and various loss functions. These experiments were also performed on code fragments ranging from 5 to 45 lines in length. The obtained results are presented in Table 4.

Table 4. Accuracy assessment of detecting heterogeneity in source codes

Method	Lines of code	Accuracy, %	Time, fragments/sec
	5	90.5	0.016
	15	91	0.045
SimNN+c_l	25	91.9	0.081
	35	93.1	0.098
	45	92.7	0.103
SimNN+t_l	5	90.4	0.012

15	91	0.042
25	91.7	0.08
35	92.5	0.091
45	93	0.099

Based on the obtained results, the most accurate approach for both cases is SimNN+c_l, with an accuracy of 94% for fragments of 25 sentences and 93% for fragments of 35 lines of code. The effectiveness of QSUM+SVM for natural language text is 8.5% lower. However, this approach operates several orders of magnitude faster and may be preferable for analyzing large volumes of textual information.

The approach based on SimNN, trained with c_l, and an ensemble of measures was employed in an experiment dedicated to determining the percentage of homogeneity in the texts of research articles and PhD dissertations. The results are presented in Table 5.

Table 5. Results of experiments on assessing the homogeneity of scientific texts

Domain	Number of co-authors							
	–	1	2	3-5	5-7	7-10	10-15	15+
	Text homogeneity, %							
Research articles	81.3	78.5	76	73.4	73.2	72.1	71	70.3
PhD dissertations	85.8	–	–	–	–	–	–	–

PhD dissertations exhibit a homogeneity rate of 85.8%. This result is due to the high originality requirements and the fact that a PhD dissertation is written by a single author. The homogeneity percentage for research articles, when authored by a single author, is 81.3%. As the number of co-authors increases, homogeneity decreases. However, for articles with five or more co-authors, the difference in homogeneity becomes negligible. This can be attributed to the fact that not all listed co-authors are directly involved in the preparation of the article's text.

A similar homogeneity assessment was conducted for source codes written by students as part of their programming practice laboratory work assignments and project-based development [28]. It should be noticed that project-based development includes more complex tasks and requires more time for its implementation. Also, project-based development is more practical and complex and emulates real-world team development because each student has a certain role and responsibilities in the project. Based on this, the hard-skill level of the project's programmers is high. The obtained results are presented in Table 6.

Table 6. Results of experiments on assessing the homogeneity of source codes

Domain	Number of students in development team						
	1	2	3	4	5	6	7
	Code homogeneity, %						

Project-based development	85,1	80,5	77,4	75	73,8	73	72,9
Practice laboratory works	88,7	–	–	–	–	–	–

Practice laboratory work exhibits a rate of 83.7%. The author's style of novice programmers is inconsistent and can improve in their learning process. However, the degree of homogeneity is influenced not only by the programmer's style but also by the functionality of the programs they develop. Practical assignments, despite the availability of numerous variations, are aimed at teaching the fundamentals of programming and involve students solving similar tasks.

In project-based development, the problems being solved are diverse, and one individual may develop modules with varying functional applications. However, the authors' styles are generally consistent. In the case of a single programmer implementing a project, the homogeneity is 85%. As the project team expands, the percentage of homogeneity decreases. With four or more participants in the project, the homogeneity percentage reaches a plateau. This phenomenon is attributed to the fact that not all members of the project team actively engage in development. Some of them focus on analytics, data research and preparation, and other secondary tasks.

The technique for authorship attribution for an open set of candidates was applied both to complete texts, in cases where the text was homogeneous, and to text fragments exhibiting signs of academic borrowing. Table 7 presents the metric values for different numbers of co-authors. For the evaluation, co-author fragments were generated using GPT models based on the styles of random authors from the dataset, taking into account the theme of the original article. A similar experiment was conducted for entirely artificially generated texts.

Table 7. Experimental results on authorship attribution of scientific texts

Domain	Number of co-authors							
	–	1	2	3-5	5-7	7-10	10-15	15+
	Accuracy, %							
Research articles	97.1	95	94.4	93	92	90.8	90.4	89
PhD dissertations	98	–	–	–	–	–	–	–
GPT-3.5	98	96.2	95	95.7	94.5	93	92.8	92.9
GPT-4	97	95.3	94.7	94	93.3	92.8	92.4	92.2

The classifier demonstrates the best discriminative ability in the absence of or with a small number of co-authors. In this case, the accuracy reaches 98%. The texts created by GPT-3.5 were the easiest to identify. Third-generation generative models exhibit a more pronounced and structured writing style compared to the latest generation.

In the context of source code authorship attribution, two approaches were tested: an ensemble of HNN, SVM, and fastText, and a CodeBERT classifier. Similar to natural language text authorship attribution, this stage involves the pre-generation of codes in

the style of random developers writing in a similar language. Additionally, experiments were conducted to identify the generative model that produced the source code.

Table 8. Experimental results on authorship attribution of source codes

Domain	Number of developers in team						
	1	2	3	4	5	6	7
	Ensemble accuracy, %						
Source codes	99	98.5	97.9	97.5	96.8	95	94.7
GPT-3.5	99.2	98.9	98.4	98.2	97.9	97.4	97
GPT-4	99	98.7	98.5	98.3	98	97.7	96.8
	CodeBERT accuracy, %						
Source codes	98.9	98.3	98	97.4	96.9	95.2	94.5
GPT-3.5	99.3	99	98.3	98.1	97.7	97.5	97
GPT-4	99	98.6	98.3	98	98	97.6	96.6

According to the *t*-test, the statistical difference between the cross-validation results obtained by the ensemble and CodeBERT is not significant: *p*-value is 0.6, which exceeds the α-value of 0.05. Based on these results, both CodeBERT and the ensemble demonstrate high effectiveness in the task of identifying the author of source code or its fragments. However, the use of the ensemble is preferable due to its lower resource requirements, as indicated by the number of model parameters – HNN, which is the basis of the ensemble, includes 1.5 million parameters, whereas CodeBERT has over 100 million. Subsequent experiments in this study were conducted using the ensemble.

In the context of experiments dedicated to source code authorship verification and identification, various approaches to contrastive learning and loss function computation were evaluated. SimNN with the loss functions *t_l* and *c_l* were employed. The evaluation was conducted on 50 random pairs of texts. These pairs could consist of texts written by different authors (author-programmers in the case of source codes) as well as texts written by the same author. The results obtained are presented in Table 9. These results were achieved with a threshold distance of 0.45 calculated using greedy search, and are presented in Table 9. search performed for each of the considered domains.

Table 9. Experimental results on various approaches to contrastive learning and loss function

Domain/Model	Triplet loss	Contrastive loss
	Accuracy, %	
Research articles	98	97.5
PhD dissertations	99.1	98
GPT-3.5, texts	99.4	99.1
GPT-4, texts	99	99
Source codes	97.5	97.4
GPT-3.5, source codes	99.2	99

GPT-4, source codes	98.6	98.5

In contrast to the results obtained during the homogeneity evaluation stage, the highest accuracy across all considered cases was achieved by SimNN trained with *t_l*. This model attained an accuracy of 98% or higher for natural language texts and 97% for artificially constructed languages.

The final experiment focused on assessing the author's contribution. According to the results obtained using the formula for calculating the author's contribution, 97% of the PhD dissertations in the dataset have an originality rate exceeding 70%.
In the case of research articles, the total contribution of each author was calculated. The number of authors identified based on heterogeneous segments (author labels) should not exceed the number of listed co-authors. The resulting percentage of works with a matching number of labels and co-authors was 87%. In research articles, the percentage of generative model labels was 9%, while in PhD dissertations, it was less than 1%.

The contribution of the programmer-author, as assessed through practice laboratory work, is 76%. This low result is attributed to the tendency of inexperienced developers to use widely adopted patterns that are most relevant based on internet search engine results. Additionally, nowadays, students often tend to use the latest generative models. The contribution of generative models is 11%.

For the case of verifying the authorship of source code authors involved in project-based development, the percentage of codes with a matching number of labels and team members was 93%. The contribution of generative models in this case was 4%.

Comparison of the obtained results with the most well-known tools for detecting plagiarism, such as «Antiplagiat», «Scribbr», and «PlagAware» is not feasible due to several factors:

- The operational principle of these tools is based on vector search across an extensive database of internal and external sources, whereas the approach proposed in the study relies on the analysis of the author's style.
- The described tools do not detect artificially generated fragments or certain model using for its creation, and therefore, they may consider such fragments original despite the incongruity with the author's style.
- The tools do not provide functionality for detecting plagiarism in artificially linguistic texts, such as source codes of programs.
- The full functionality of these systems is available only through paid subscriptions.

Thus, the proposed technique is more universal and effective for addressing specific applied tasks related to the detection of plagiarism and the use of generative models in the creation of texts/programs in academic and educational environments.

6 Conclusion

As part of this research, a technique for checking homogeneity and detecting plagiarism was developed, distinguished by the combination of SimNN and specific loss functions. This technique enables the identification of heterogeneity, plagiarism, and fact of using

artificially generated texts, as well as effectively verifying the authorship of two or more texts.

The experiment assessing homogeneity revealed that the number of co-authors significantly affects the number of heterogeneous fragments. The average homogeneity of dissertations in the dataset is 85.8%, scientific articles range from 70.3% to 81.3% depending on the number of co-authors, and source codes range from 72.9% to 85.1%.

The technique was applied to the dataset of scientific texts to identify the percentage of heterogeneous, plagiarized, and artificially generated fragments. The percentage of PhD dissertations with originality exceeding the 70% threshold is 97% (with 1% of the plagiarism attributed to artificial generation). For scientific articles, the originality percentage is 87% (with 9% of artificial generation). The technique was also tested on source code samples. The originality percentage for students' practical programming assignments is 97.5% (with 11% being artificially generated), and for group project-based development, it is 93% (with 4% being artificially generated).

This research was funded by the Ministry of Science and Higher Education of Russia, Government Order for 2023–2025, project no. FEWM-2023-0015 (TUSUR).

References

1. Baria, H.; Bamaniya, M.; Mahato, T.; Suthar, D.; Kushvah R. PLAGIARISM - AN ACT AGAINST THE RESEARCH ETHICS. *Indian Journal of Scientific Research*. **2023**, *3*, 114-119.
2. Scribbr. Available online: https://www.scribbr.com (accessed on 31 July 2024).
3. PlagAware. Available online: https://www.plagaware.com/ (accessed on 31 July 2024).
4. Antiplagiat. Available online: https://antiplagiat.ru/en/ (accessed on 31 July 2024).
5. Fanshawelibrary. Available online: https://www.fanshawelibrary.com (accessed on 31 July 2024).
6. n'RIS. Available online: https://nris.ru/ (accessed on 31 July 2024).
7. GPTZero. Available online: https://gptzero.me/ (accessed on 31 July 2024).
8. Writer - AI content detector. Available online: https://writer.com/ai-content-detector/ (accessed on 31 July 2024).
9. Moravvej, S.V.; Mousavirad, S.J.; Oliva, D.; Mohammadi, F. A Novel Plagiarism Detection Approach Combining BERT-based Word Embedding, Attention-based LSTMs and an Improved Differential Evolution Algorithm. *arXiv preprint.* **2023**. arXiv:2305.02374.
10. Sanborn, A., Skryzalin J. Deep learning for semantic similarity. In CS224d: Deep Learning for Natural Language Processing. Stanford University Stanford, CA, USA, **2015**.
11. Pontes, E.L.; Huet, S.; Linhares, A.; Torres-Moreno J-M. Predicting the Semantic Textual Similarity with Siamese CNN and LSTM. *arXiv preprint.* **2018.** arXiv:1810.10641.
12. Bao, W; Bao, W.; Du, J.; Yang, Y.; Zhao, X. Attentive siamese LSTM network for semantic textual similarity measure. In International Conference on Asian Language Processing, 2018; pp. 312–317.
13. El-Rashidy, M.A.; Mohamed, R.G.; El-Fishawy, N.A. et al. An effective text plagiarism detection system based on feature selection and SVM techniques. *Multimed Tools Appl,* **2024,** *83*, 2609–2646.
14. El Mostafa, H.; Faouzia, B. A New Online Plagiarism Detection System based on Deep Learning. *International Journal of Advanced Computer Science and Applications(IJACSA)*, **2020,** *11(9)*, 470-478.

15. Ahuja, L.; Gupta, V.; Kumar, R. A New Hybrid Technique for Detection of Plagiarism from Text Documents. *Arab J Sci Eng 45*, **2020**, 9939–9952.
16. Ebrahim, F.; Joy, M.Source Code Plagiarism Detection with Pre-Trained Model Embeddings and Automated Machine Learning. Proceedings of Recent Advances in Natural Language Processing, Varna, Sep 4–6, 2023; pp. 301-309.
17. Fedotova, A.; Romanov, A.; Kurtukova, A.; Shelupanov, A. Authorship Attribution of Social Media and Literary Russian-Language Texts Using Machine Learning Methods and Feature Selection. *Future Internet* **2022**, *14*, 4.
18. Fedotova, A.; Romanov, A.; Kurtukova, A.; Shelupanov, A. Digital Authorship Attribution in Russian-Language Fanfiction and Classical Literature. *Algorithms* **2023**, *16*, 13.
19. Romanov, A.; Kurtukova, A.; Fedotova, A.; Shelupanov, A. Authorship Identification of Binary and Disassembled Codes Using NLP Methods. *Information* **2023**, *14*, 361.
20. Kurtukova, A.; Romanov, A.; Shelupanov, A. Source Code Authorship Identification Using Deep Neural Networks. *Symmetry* **2020**, *12*, 2044.
21. OpenAI - GPT-3.5. Available online: https://openai.com/index/gpt-3-5-turbo-fine-tuning-and-api-updates/ (accessed on 31 July 2024).
22. OpenAI - GPT-4. Available online: https://openai.com/index/gpt-4/ (accessed on 31 July 2024).
23. Romanov A.S., Rezanova Z.I., Meshcheryakov R.V. Methodology for checking text homogeneity and identifying plagiarism based on the support vector machine and fast correlation filter. *Doklady TUSUR*. **2014**. *2(32)*, 264–269 (In Russian).
24. Le-Khac, P.H.; Healy, G.; Smeaton, A.F. Contrastive Representation Learning: A Framework and Review. *arXiv preprint*. **2020**. arXiv:2010.05113.
25. Neculoiu, P.; Versteegh, M.; Rotaru, M. Learning Text Similarity with Siamese Recurrent Networks. Proceedings of the 1st Workshop on Representation Learning for NLP, Berlin, Germany, August 11, 2016; pp. 148–157.
26. Amer, A.; Nguyen, L. Combinations of Jaccard with Numerical Measures for Collaborative Filtering Enhancement: Current Work and Future Proposal. *arXiv preprint*. **2021**. arXiv:2111.12202.
27. Caliskan-Islam, A. Deanonymizing programmers via code stylometry. In Proceedings of the 24th USENIX Security Symposium, Washington, DC, USA, 12–14 August 2015; pp. 255–270.